DAX Conditional Logic, Multi-Condition Checks & Calculated Columns in Power BI

1. Introduction to Conditional Logic in DAX

In Power BI, conditions are evaluated using **logical functions** such as:

- **IF()**
- **IFS()**
- **AND()**
- **OR()**
- **SWITCH()**
- **Comparison operators** (>, <, >=, <=, =, <>)

YATICORP
EMPOWERING EDUCATION

These functions are used inside:

- **Calculated Columns**
- **Measures**
- **Calculated Tables**

In your narrative, you were classifying values into **High**, **Medium**, and **Low** categories using conditional checks based on **Revenue** and **Profit**, and using operators like **OR** and **AND**.

Let’s break this down in an easy and comprehensive manner.

---

2. Calculated Columns for Conditional Segmentation

✔ What is a Calculated Column?

A **Calculated Column** is a DAX expression that is evaluated **row by row** during data refresh.
It becomes a part of the data model and each row has a fixed value.

Example:

```
Value Segment = IF(Sales[Revenue] > 2000, "High", "Low")
```

✔ When to use Calculated Columns?

- When the result must exist physically in the table
- When logic is evaluated row-wise
- When the result does not depend on filter context
- Useful for labels, categories, and classifications (like your example)

YATICORP
EMPOWERING EDUCATION

---

3. Using IF() for Conditional Classification

Syntax of IF()

```
IF(
   logical_test,
   result_if_true,
   [result_if_false]
)
```

Single Condition Example

```
Value Segment = IF(SUM(Sales[Revenue]) > 2000, "High Value", "Low Value")
```

However, **SUM()** cannot be used inside a calculated column for row-wise logic.
For row-level checks, you must use the column directly:

```
Value Segment = IF(Sales[Revenue] > 2000, "High Value", "Low Value")
```

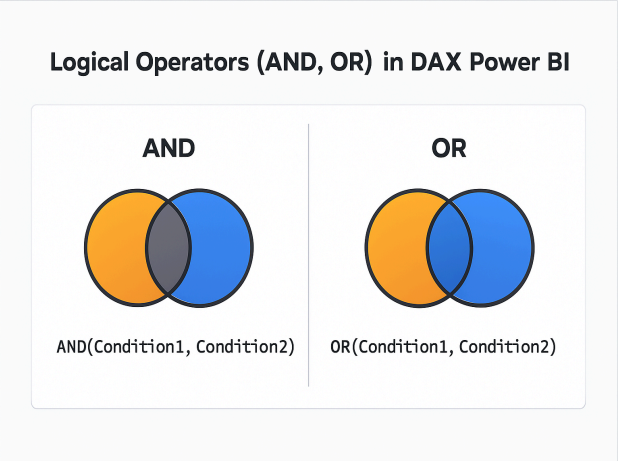
---

4. Logical Operators (AND, OR)

4.1 Using OR() for multiple possible conditions

You used OR to say:

> If Revenue > 2000 **OR** Profit > 100 then return "High Value"




```
Value Segment OR =
IF(
   OR(Sales[Revenue] >= 2000, Sales[Profit]
>= 100),
   "High Value",
   "Not High"
)
```

✔ OR is used when any one of the conditions must be true.

YATICORP
EMPOWERING EDUCATION

4.2 Using AND() for stricter logic

You later created another column where **both** conditions must be true:

> Revenue >= 2000 **AND** Profit >= 100 → Medium / High Value

```
Value Segment AND =
IF(
   AND(Sales[Revenue] >= 2000, Sales[Profit] >= 100),
   "Medium Value",
   "Other"
)
```

✔ AND is used when all conditions must be true.

---

5. Using Nested IF() for Multiple Conditional Checks

If you want more than one condition like:

- HIGH when Revenue >= 3000 or Profit >= 200
- MEDIUM when Revenue >= 2000 and Profit >= 100
- LOW otherwise

You may use **nested IF statements**:

```
Value Segment Nested =
IF(
   OR(Sales[Revenue] >= 3000, Sales[Profit] >= 200),
   "High Value",
   IF(
      AND(Sales[Revenue] >= 2000, Sales[Profit] >= 100),
      "Medium Value",
      "Low Value"
   )
)
```

YATICORP
EMPOWERING EDUCATION

✔ **Nesting IF works but becomes difficult to maintain as conditions increase.**

---

6. Using IFS() for Multiple Conditions Without Nesting

Power BI provides a cleaner alternative: **IFS()**

Syntax

```
IFS(
   condition1, result1,
   condition2, result2,
   condition3, result3
)
```

Example using your revenue and profit logic

```
Value Segment IFS =
IFS(
    Sales[Revenue] >= 3000 || Sales[Profit] >= 200, "High Value",
    Sales[Revenue] >= 2000 && Sales[Profit] >= 100, "Medium Value",
    TRUE(), "Low Value"
)
```

✔ **IFS reduces complexity and improves readability.**

✔ **The final TRUE() acts like an ELSE condition.**

---

7. Why SWITCH() Is Better for Large Category Lists

You mentioned that instead of 4 regions, imagine **50 regions**.

Creating IF or IFS checks like:

```
IF(Region = "East", ..., IF(Region = "West", ...
```

becomes extremely messy.

This is where the SWITCH() function is powerful.

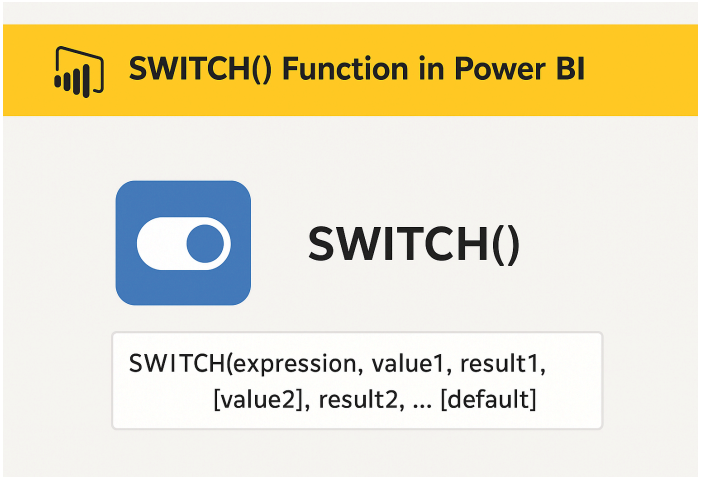
YATICORP
EMPOWERING EDUCATION

---

8. SWITCH() Function – Cleaner Alternative to IF/IFS

Syntax 1: SWITCH with Expression Matching

```
SWITCH(expression,
    value1, result1,
    value2, result2,
    value3, result3,
    elseResult
)
```



Example: Region classification

```
Region Category =
SWITCH(Sales[Region],
    "East", "Zone 1",
    "West", "Zone 2",
    "North", "Zone 3",
    "South", "Zone 4",
    "Other"
)
```

✔ **Best when you match exact values.**

✔ **More readable than nested IF.**

---

9. SWITCH(TRUE()) – The Most Powerful Form

This is extremely useful when you want to evaluate **ranges** or **multiple conditions** based on logical expressions.

YATICORP
EMPOWERING EDUCATION

Example (your scenario with Revenue & Profit)

```
Value Segment Switch =
SWITCH(
    TRUE(),
    Sales[Revenue] >= 3000 || Sales[Profit] >= 200, "High Value",
    Sales[Revenue] >= 2000 && Sales[Profit] >= 100, "Medium Value",
    Sales[Revenue] < 2000 && Sales[Profit] < 100, "Low Value",
    "Other"
)
```

✔ **SWITCH(TRUE()) works like an advanced IF-ELSE ladder**

✔ **Improves readability**

✔ **Very useful for segmentation logic**

---

10. Practical Example Considering Your Explanation

Let's rebuild your full logic in a clear structure.

Column 1: OR-based high-value segment

```
High Segment =
IF(
    OR(Sales[Revenue] >= 2000, Sales[Profit] >= 100),
    "High Value",
    "Not High"
)
```

Column 2: AND-based medium-value segment

```
Medium Segment =
IF(
    AND(Sales[Revenue] >= 2000, Sales[Profit] >= 100),
    "Medium Value",
    "Not Medium"
)
```

YATICORP
EMPOWERING EDUCATION

Final Column Using SWITCH (Best Approach)

```
Value Segment =
SWITCH(
    TRUE(),
    OR(Sales[Revenue] >= 3000, Sales[Profit] >= 200), "High Value",
    AND(Sales[Revenue] >= 2000, Sales[Profit] >= 100), "Medium Value",
    Sales[Revenue] < 2000, "Low Value",
    "Other"
)
```

This is neat, efficient, and clean.

---

11. Performance Comparison: IF vs IFS vs SWITCH

IF

- Works for 1–2 conditions
- Becomes messy when nested
- Prone to errors

IFS

- Eliminates nesting
- Best for 3–5 conditions
- Requires conditions to be ordered correctly
- No built-in ELSE; must use TRUE()

SWITCH

- Perfect for categorical mapping
- Best readability
- SWITCH(TRUE()) is most powerful for numeric ranges
- High performance because evaluation stops once a match is found

In complex business logic, SWITCH(TRUE()) is preferred.

YATICORP
EMPOWERING EDUCATION

12. When to Use AND/OR

✔ Use OR when:

You want ANY condition to be true
Example:

```
OR(A > 1000, B > 100)
```

✔ Use AND when:

You want ALL conditions to be true
Example:

```
AND(A > 1000, B > 100)
```

✔ Combine them for advanced logic

Example:

```
A > 1000 && (B > 500 || C = "USA")
```

YATICORP
EMPOWERING EDUCATION

---

13. Practical Industry Examples

13.1 Customer Segmentation

```
Customer Tier =
SWITCH(
    TRUE(),
    Sales[Revenue] > 20000, "Platinum",
    Sales[Revenue] > 10000, "Gold",
    Sales[Revenue] > 5000, "Silver",
    "Bronze"
)
```

13.2 Product Rating Based on Two KPIs

```
Product Score =
SWITCH(
    TRUE(),
    AND(ProfitMargin > 0.3, Rating > 4.5), "Premium",
    AND(ProfitMargin > 0.2, Rating > 4.0), "Above Average",
    AND(ProfitMargin > 0.1, Rating > 3.0), "Standard",
    "Low"
)
```

13.3 Age Groups

```
Age Group =
SWITCH(
    TRUE(),
    Age >= 60, "Senior",
    Age >= 30, "Adult",
    Age >= 13, "Teen",
    "Child"
)
```

YATICORP
EMPOWERING EDUCATION

---

14. Advantages of Using SWITCH() Over IF

Feature	IF	SWITCH
Readability	Poor when nested	Excellent
Maintenance	Hard	Easy
Performance	Slower for many conditions	Faster
Mapping large sets	Difficult	Very easy
Checks multiple conditions	Yes (nested)	Yes (with TRUE())

---

15. Common Errors and How to Avoid Them

❌ Using SUM() inside calculated columns

Calculated columns operate row-wise; SUM() is for aggregations across rows.

Correct approach:
Use the column directly:

```
Sales[Revenue]
```

❌ Incorrect ordering of IFS conditions

Always write the **highest priority** condition first.

❌ Missing ELSE condition in SWITCH or IFS

Always include:

```
TRUE(), "Other"
```

YATICORP
EMPOWERING EDUCATION

---

16. Summary: Best Practices for Multi-Condition Logic in DAX

- Use **IF** for simple, single conditions
- Use **IFS** for multiple linear conditions
- Use **AND/OR** for combining logical rules
- Use **SWITCH** for matching specific values
- Use **SWITCH(TRUE())** for complex segmentation
- Avoid nested IFs where possible
- Prefer calculated columns for static classification
- Prefer measures for dynamic, filter-dependent calculations

---

17. Final Combined Example (Best Practice)

```
Value Segment Final =
SWITCH(
    TRUE(),
    Sales[Revenue] >= 3000 || Sales[Profit] >= 200, "High Value",
    Sales[Revenue] >= 2000 && Sales[Profit] >= 100, "Medium Value",
    Sales[Revenue] < 2000 && Sales[Profit] < 100, "Low Value",
    "Other"
)
```

This version is clean, powerful, efficient, and easy to maintain.

YATICORP
EMPOWERING EDUCATION